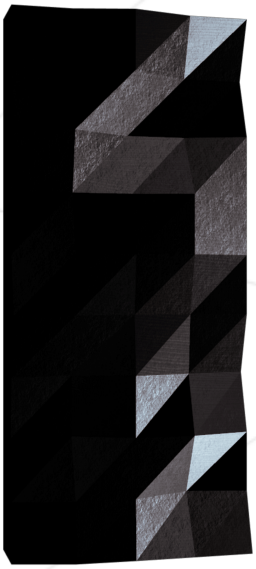

C++ - Module 01

Memory allocation, pointers to members, references and switch statements

Summary:
This document contains the exercises of Module 01 from C++ modules.

Version: 11.1

Contents

I	**Introduction**	**2**
II	**General rules**	**3**
III	**AI Instructions**	**6**
IV	**Exercise 00: BraiiiiiiinnnzzzZ**	**8**
V	**Exercise 01: Moar brainz!**	**9**
VI	**Exercise 02: HI THIS IS BRAIN**	**10**
VII	**Exercise 03: Unnecessary violence**	**11**
VIII	**Exercise 04: Sed is for losers**	**13**
IX	**Exercise 05: Harl 2.0**	**14**
X	**Exercise 06: Harl filter**	**16**
XI	**Submission and peer-evaluation**	**17**

Chapter I

Introduction

C++ is a general-purpose programming language created by Bjarne Stroustrup as an extension of the C programming language, or "C with Classes" (source: Wikipedia).

The goal of these modules is to introduce you to **Object-Oriented Programming**. This will be the starting point of your C++ journey. Many languages are recommended for learning OOP. We have chosen C++ since it is derived from your old friend, C. Because this is a complex language, and in order to keep things simple, your code will comply with the C++98 standard.

We are aware that modern C++ is very different in many aspects. So if you want to become a proficient C++ developer, it is up to you to go further after the 42 Common Core!

Chapter II

General rules

Compiling

- Compile your code with `c++` and the flags `-Wall -Wextra -Werror`
- Your code should still compile if you add the flag `-std=c++98`

Formatting and naming conventions

- The exercise directories will be named this way: `ex00, ex01, ...  , exn`
- Name your files, classes, functions, member functions and attributes as required in the guidelines.
- Write class names in **UpperCamelCase** format. Files containing class code will always be named according to the class name. For instance: `ClassName.hpp/ClassName.h`, `ClassName.cpp`, or `ClassName.tpp`. Then, if you have a header file containing the definition of a class "BrickWall" standing for a brick wall, its name will be `BrickWall.hpp`.
- Unless specified otherwise, every output message must end with a newline character and be displayed to the standard output.
- *Goodbye Norminette!* No coding style is enforced in the C++ modules. You can follow your favorite one. But keep in mind that code your peer evaluators can't understand is code they can't grade. Do your best to write clean and readable code.

Allowed/Forbidden

You are not coding in C anymore. Time to C++! Therefore:

- You are allowed to use almost everything from the standard library. Thus, instead of sticking to what you already know, it would be smart to use the C++-ish versions of the C functions you are used to as much as possible.
- However, you can't use any other external library. It means C++11 (and derived forms) and `Boost` libraries are forbidden. The following functions are forbidden too: `*printf()`, `*alloc()` and `free()`. If you use them, your grade will be 0 and that's it.

- Note that unless explicitly stated otherwise, the `using namespace <ns_name>` and `friend` keywords are forbidden. Otherwise, your grade will be -42.
- **You are allowed to use the STL only in Modules 08 and 09.** That means: no **Containers** (vector/list/map, and so forth) and no **Algorithms** (anything that requires including the `<algorithm>` header) until then. Otherwise, your grade will be -42.

A few design requirements

- Memory leakage occurs in C++ too. When you allocate memory (by using the `new` keyword), you must avoid **memory leaks**.
- From Module 02 to Module 09, your classes must be designed in the **Orthodox Canonical Form, except when explicitly stated otherwise**.
- Any function implementation put in a header file (except for function templates) means 0 to the exercise.
- You should be able to use each of your headers independently from others. Thus, they must include all the dependencies they need. However, you must avoid the problem of double inclusion by adding **include guards**. Otherwise, your grade will be 0.

Read me

- You can add some additional files if you need to (i.e., to split your code). As these assignments are not verified by a program, feel free to do so as long as you turn in the mandatory files.
- Sometimes, the guidelines of an exercise look short but the examples can show requirements that are not explicitly written in the instructions.
- Read each module completely before starting! Really, do it.
- By Odin, by Thor! Use your brain!!!

Regarding the Makefile for C++ projects, the same rules as in C apply (see the Norm chapter about the Makefile).

You will have to implement a lot of classes. This can seem tedious, unless you're able to script your favorite text editor.

You are given a certain amount of freedom to complete the exercises. However, follow the mandatory rules and don't be lazy. You would miss a lot of useful information! Do not hesitate to read about theoretical concepts.

Chapter III

AI Instructions

Context

This project is designed to help you discover the fundamental building blocks of your 42 training.

To properly anchor key knowledge and skills, it's essential to adopt a thoughtful approach to using AI tools and support.

True foundational learning requires genuine intellectual effort — through challenge, repetition, and peer-learning exchanges.

For a more complete overview of our stance on AI — as a learning tool, as part of the 42 training, and as an expectation in the job market — please refer to the dedicated FAQ on the intranet.

Main message

- Build strong foundations without shortcuts.
- Really develop tech & power skills.
- Experience real peer-learning, start learning how to learn and solve new problems.
- The learning journey is more important than the result.
- Learn about the risks associated with AI, and develop effective control practices and countermeasures to avoid common pitfalls.

Learner rules:

- You should apply reasoning to your assigned tasks, especially before turning to AI.

- You should not ask for direct answers to the AI.
- You should learn about 42 global approach on AI.

Phase outcomes:

Within this foundational phase, you will get the following outcomes:

- Get proper tech and coding foundations.
- Know why and how AI can be dangerous during this phase.

Comments and example:

- Yes, we know AI exists — and yes, it can solve your projects. But you're here to learn, not to prove that AI has learned. Don't waste your time (or ours) just to demonstrate that AI can solve the given problem.
- Learning at 42 isn't about knowing the answer — it's about developing the ability to find one. AI gives you the answer directly, but that prevents you from building your own reasoning. And reasoning takes time, effort, and involves failure. The path to success is not supposed to be easy.
- Keep in mind that during exams, AI is not available — no internet, no smartphones, etc. You'll quickly realise if you've relied too heavily on AI in your learning process.
- Peer learning exposes you to different ideas and approaches, improving your interpersonal skills and your ability to think divergently. That's far more valuable than just chatting with a bot. So don't be shy — talk, ask questions, and learn together!
- Yes, AI will be part of the curriculum — both as a learning tool and as a topic in itself. You'll even have the chance to build your own AI software. In order to learn more about our crescendo approach you'll go through in the documentation available on the intranet.

✓ Good practice:

> I'm stuck on a new concept. I ask someone nearby how they approached it. We talk for 10 minutes — and suddenly it clicks. I get it.

✗ Bad practice:

> I secretly use AI, copy some code that looks right. During peer evaluation, I can't explain anything. I fail. During the exam — no AI — I'm stuck again. I fail.

Chapter IV

Exercise 00: BraiiiiiiinnnzzzZ

	Exercise: 00
BraiiiiiiinnnzzzZ	
Directory: $ex00/$	
Files to Submit: `Makefile, main.cpp, Zombie.{h, hpp}, Zombie.cpp, newZombie.cpp, randomChump.cpp`	
Forbidden: `None`	

First, implement a **Zombie** class. It has a private string attribute `name`.

Add a member function `void announce( void );` to the Zombie class. Zombies announce themselves as follows:

```
<name>: BraiiiiiiinnnzzzZ...
```

Do not print the angle brackets (< and >). For a zombie named Foo, the message would be:

```
Foo: BraiiiiiiinnnzzzZ...
```

Then, implement the following two functions:

- `Zombie* newZombie( std::string name );`
  This function creates a zombie, names it, and returns it so you can use it outside of the function scope.
- `void randomChump( std::string name );`
  This function creates a zombie, names it, and makes it announce itself.

Now, what is the actual point of the exercise? You have to determine in which case it is better to allocate zombies on the stack or the heap.

Zombies must be destroyed when you no longer need them. The destructor must print a message with the name of the zombie for debugging purposes.

Chapter V

Exercise 01: Moar brainz!

	Exercise: 01
Moar brainz!	
Directory: $ex01/$	
Files to Submit: `Makefile, main.cpp, Zombie.{h, hpp}, Zombie.cpp, zombieHorde.cpp`	
Forbidden: `None`	

Time to create a **horde of Zombies**!

Implement the following function in the appropriate file:

```
Zombie* zombieHorde( int N, std::string name );
```

It must allocate `N` Zombie objects in a single allocation. Then, it must initialize the zombies, giving each of them the name passed as a parameter. The function returns a pointer to the first zombie.

Implement your own tests to ensure that your `zombieHorde()` function works as expected. Try calling `announce()` for each of the zombies.

Do not forget to use `delete` to deallocate all the zombies and check for **memory leaks**.

Chapter VI

Exercise 02: HI THIS IS BRAIN

	Exercise: 02
HI THIS IS BRAIN	
Directory: $ex02/$	
Files to Submit: `Makefile, main.cpp`	
Forbidden: `None`	

Write a program that contains:

- A string variable initialized to "HI THIS IS BRAIN".
- `stringPTR`: a pointer to the string.
- `stringREF`: a reference to the string.

Your program must print:

- The memory address of the string variable.
- The memory address held by `stringPTR`.
- The memory address held by `stringREF`.

And then:

- The value of the string variable.
- The value pointed to by `stringPTR`.
- The value pointed to by `stringREF`.

That's all—no tricks. The goal of this exercise is to demystify references, which may seem completely new. Although there are some small differences, this is simply another syntax for something you already do: address manipulation.

Chapter VII

Exercise 03: Unnecessary violence

	Exercise: 03
	Unnecessary violence
Directory: $ex03/$	
Files to Submit: `Makefile, main.cpp, Weapon.{h, hpp}, Weapon.cpp, HumanA.{h, hpp}, HumanA.cpp, HumanB.{h, hpp}, HumanB.cpp`	
Forbidden: `None`	

Implement a **Weapon** class that has:

- A private attribute `type`, which is a string.
- A `getType()` member function that returns a constant reference to `type`.
- A `setType()` member function that sets `type` using the new value passed as a parameter.

Now, create two classes: **HumanA** and **HumanB**. They both have a `Weapon` and a `name`. They also have a member function `attack()` that displays (without the angle brackets):

```
<name> attacks with their <weapon type>
```

HumanA and HumanB are almost identical except for these two small details:

- While **HumanA** takes the `Weapon` in its constructor, **HumanB** does not.
- **HumanB** may **not always** have a weapon, whereas **HumanA** will **always** be armed.

If your implementation is correct, executing the following code will print an attack with "crude spiked club" followed by a second attack with "some other type of club" for both test cases:

```
int main()
{
  {
    Weapon  club = Weapon("crude spiked club");

    HumanA bob("Bob", club);
    bob.attack();
    club.setType("some other type of club");
    bob.attack();
  }
  {
    Weapon  club = Weapon("crude spiked club");

    HumanB jim("Jim");
    jim.setWeapon(club);
    jim.attack();
    club.setType("some other type of club");
    jim.attack();
  }

  return 0;
}
```

Do not forget to check for **memory leaks**.

In which case do you think it would be best to use a pointer to Weapon? And a reference to Weapon? Why? Think about it before starting this exercise.

Chapter VIII

Exercise 04: Sed is for losers

	Exercise: 04
Sed is for losers	
Directory: *ex*04/	
Files to Submit: `Makefile, main.cpp, *.cpp, *.{h, hpp}`	
Forbidden: `std::string::replace`	

Create a program that takes three parameters in the following order: a filename and two strings, `s1` and `s2`.

It must open the file `<filename>` and copy its content into a new file `<filename>.replace`, replacing every occurrence of `s1` with `s2`.

Using C file manipulation functions is forbidden and will be considered cheating. All the member functions of the class `std::string` are allowed, except `replace`. Use them wisely!

Of course, handle unexpected inputs and errors. You must create and turn in your own tests to ensure that your program works as expected.

Chapter IX

Exercise 05: Harl 2.0

	Exercise: 05
Harl 2.0	
Directory: *ex*05/	
Files to Submit: `Makefile, main.cpp, Harl.{h, hpp}, Harl.cpp`	
Forbidden: `None`	

Do you know Harl? We all do, don't we? In case you don't, find below the kind of comments Harl makes. They are classified by levels:

- "**DEBUG**" level: Debug messages contain contextual information. They are mostly used for problem diagnosis.
  Example: *"I love having extra bacon for my 7XL-double-cheese-triple-pickle-special-ketchup burger. I really do!"*
- "**INFO**" level: These messages contain extensive information. They are helpful for tracing program execution in a production environment.
  Example: *"I cannot believe adding extra bacon costs more money. You didn't put enough bacon in my burger! If you did, I wouldn't be asking for more!"*
- "**WARNING**" level: Warning messages indicate a potential issue in the system. However, it can be handled or ignored.
  Example: *"I think I deserve to have some extra bacon for free. I've been coming for years, whereas you started working here just last month."*
- "**ERROR**" level: These messages indicate that an unrecoverable error has occurred. This is usually a critical issue that requires manual intervention.
  Example: *"This is unacceptable! I want to speak to the manager now."*

You are going to automate Harl. It won't be difficult since he always says the same things. You have to create a **Harl** class with the following private member functions:

- `void debug( void );`
- `void info( void );`
- `void warning( void );`
- `void error( void );`

Harl also has a public member function that calls the four member functions above depending on the level passed as a parameter:

```
void    complain( std::string level );
```

The goal of this exercise is to use **pointers to member functions**. This is not a suggestion. Harl has to complain without using a forest of if/else if/else. He doesn't think twice!

Create and turn in tests to show that Harl complains a lot. You can use the examples of comments listed above in the subject or choose to use comments of your own.

Chapter X

Exercise 06: Harl filter

	Exercise: 06
Harl filter	
Directory: *ex*06/	
Files to Submit: `Makefile, main.cpp, Harl.{h, hpp}, Harl.cpp`	
Forbidden: `None`	

Sometimes you don't want to pay attention to everything Harl says. Implement a system to filter what Harl says depending on the log levels you want to listen to.

Create a program that takes as a parameter one of the four levels. It will display all messages from this level and above. For example:

```
$> ./harlFilter "WARNING"
[ WARNING ]
I think I deserve to have some extra bacon for free.
I've been coming for years, whereas you started working here just last month.

[ ERROR ]
This is unacceptable! I want to speak to the manager now.

$> ./harlFilter "I am not sure how tired I am today..."
[ Probably complaining about insignificant problems ]
```

Although there are several ways to deal with Harl, one of the most effective is to SWITCH it off.

Give the name `harlFilter` to your executable.

You must use, and maybe discover, the switch statement in this exercise.

`You can pass this module without doing exercise 06.`

Chapter XI

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double-check the names of your folders and files to ensure they are correct.

During the evaluation, a brief **modification of the project** may occasionally be requested. This could involve a minor behaviour change, a few lines of code to write or rewrite, or an easy-to-add feature.

While this step may **not be applicable to every project**, you must be prepared for it if it is mentioned in the evaluation guidelines.

This step is meant to verify your actual understanding of a specific part of the project. The modification can be performed in any development environment you choose (e.g., your usual setup), and it should be feasible within a few minutes — unless a specific time frame is defined as part of the evaluation.
You can, for example, be asked to make a small update to a function or script, modify a display, or adjust a data structure to store new information, etc.

The details (scope, target, etc.) will be specified in the **evaluation guidelines** and may vary from one evaluation to another for the same project.